

## Elección PK

↳ Horrible ID automático  
(Inconcebible)

- Unicidad (→ si es inconcebible  
(+ importante) que a día de hoy hay duplicados  
insuficiente P/  
claridad unicidad)

- siempre  $\exists$

- Conocido

- No cambia (Inmutable)

- Clave natural

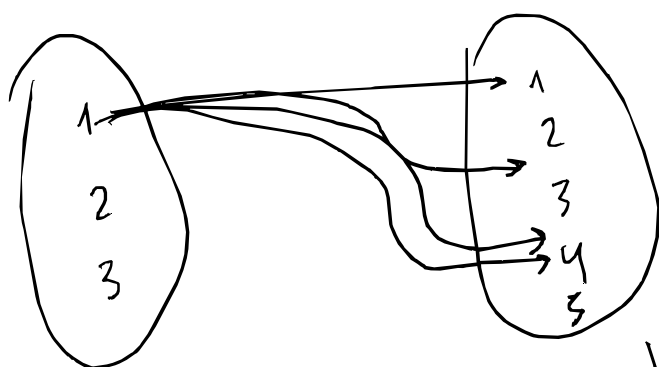
(existe permanentemente algo  
que pertenece a la entidad real)

↳ ayuda a mantener  
el riesgo

(vs AUTOINCREMENT ID)

↳ provee  $\oplus$  simplicidad  
 $\oplus$  semántico

## Relación (modelo relacional)



$\langle 1, 1 \rangle$   
 $\langle 1, 2 \rangle$   
 $\langle 1, 3 \rangle$

1 es divisor de 1,  
2,  
3,  
4

## Modelo relacional

↳ expresado como conjuntos

⇒ hereda todas las formulas

operación

propiedades de teoría de conjuntos

⇒ más p/ mejorar la  
ejecución de las queries  
(equivale a p/ de  
conjuntos)

no demostrado

# Formas Normal

→ non, strings, etc

1FN → todos atributos atómicos  
↓ otherwise  
lores consistency

2FN → otr no primas → dependen  
de PK completo

→ dependencia funcional  
X puede conocer uno  
como respecto de lo otro  
(deducible X de Y)

3FN →  $\nexists$  DF transitive en  
los atributos que no son  
clave.

Por ejemplo:

registro de factura  
deberio duplicar precio p/  
registro el precio del  
producto, independiente  
del cambio de precio futuro

BCFN → c/determinante

→ atributo que  
determina completamente  
a otro

→ es clave candidato

Objetivo BD: Operaciones rápidas

RAM > es que sin esfuerzo  
se pueda acceder

FNS  
Importante ACID p/ primer  
Concurrencia

EVITAR REDUNDANCIA  
DE INFORMACIÓN

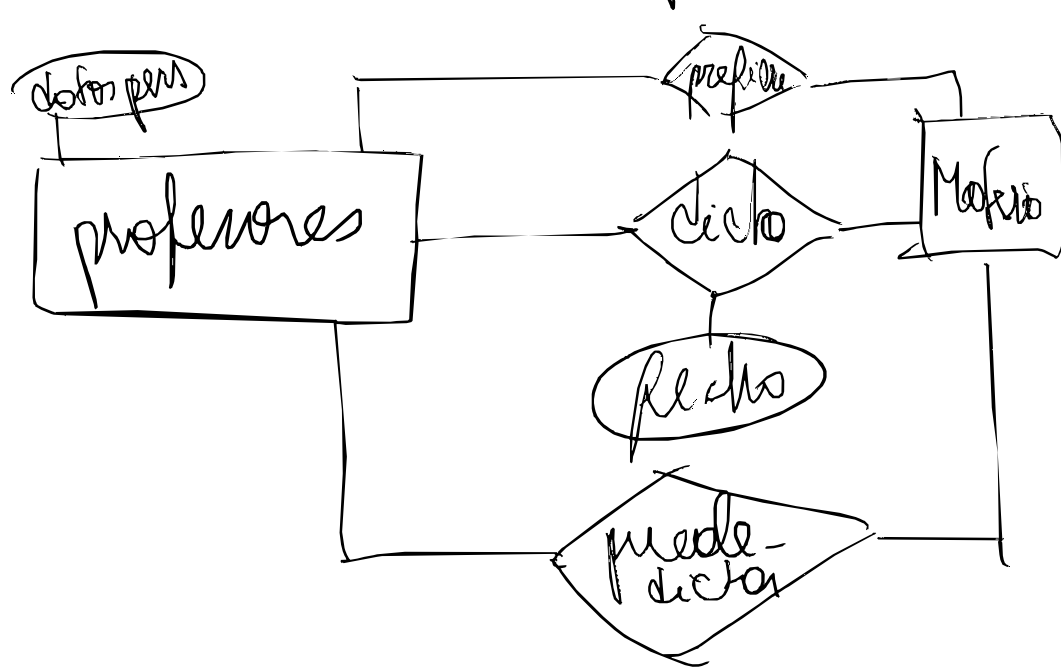
problemas  
inconsistency  
⊕  
discrepancy

Dependencia funcional  $\rightarrow$  representa dependencias en los records

$\downarrow$   
 $\forall t_1, t_2$  (tuplas), Si  $t_1[x] = t_2[x]$   
 $\downarrow$   
 $t_1[y] = t_2[y]$   
 inconcebible que no suceda

CK = clave candidato for a ser PK

- $\rightarrow$  minimal
- $\rightarrow$  primero determino la relación completa



Tuplas espurias / Atributos espurios

$\downarrow$

idea: puede ser atributos y tener una tabla válida más chico

create table horarios (... , primary key (id mat, doc, hini))

constraints  
 hooks for CK

[ unique (id mat, doc, hini)  
 :  
 :  
 :  
 check (hIni > 0)  
 :  
 :  
 :

- SQL Triggers: PL code for complex constraints  $\rightarrow$  not backwards compatible
- Data Types: Range, GIS, etc.  $\downarrow$  (postgres)   
 triggered by table modify
- A veces es necesario romper algunos FN p/ lograr representar lo que necesita
- ON-UPDATE CASCADE  $\rightarrow$  update is reflected in all references to table
- function types sql  $\rightarrow$  immutable, stable, volatile
- FK pueden referenciar PK o uniques
- NULL = ignorado, no sé,  $\perp$ 
  - $\rightarrow$  is changes semantic of null  $\rightarrow$  null = empty.
  - $\rightarrow$  aplica comp lógica trivializada p/ operaciones lógicas

# P/ final sobre el libro de ULLMAN (Vol 2)

---

## Planes de ejecución (ver en Ullman)

ENULLMAN: QUERY EXECUTION 201, <sup>Physical Query</sup> <sup>Plan Operators</sup>

Error común → Delete takes  
too long

↳ A is FK on  
other table  
but said table  
doesn't have an  
index for A  
∴ full scan  
for dependencies

- uso el índice solo si  
me conviene → <sup>nó lo</sup> <sup>tabla</sup>
- tiene datos estadísticos  $\Delta$  ANALYZE
- Buckets?
- de intento de manipular la  
tabla es un chico y no  
lo prueba
- índices son importantes
- planes de ejecución  
↳ un SQL podría estar pulido  
pero podría no funcionar xq  
el motor de la DB puede  
tomar decisiones malas